Problem 5.7.3

Sarvesh Tamgade

September 17, 2025

Question

Question: For the matrix

$$\begin{pmatrix} 1 & 1 & 1 \\ 1 & 2 & -2 \\ 2 & -1 & 3 \end{pmatrix},$$

show that

$$A^3 - 6A^2 + 5A + 11I = 0. \tag{1.1}$$

Find A^{-1}.

Solution

Solution: The characteristic equation is

$$\left|\begin{pmatrix} 1-\lambda & 1 & 1 \\ 1 & 2-\lambda & -2 \\ 2 & -1 & 3-\lambda \end{pmatrix}\right| = 0, \tag{2.1}$$

which expands to

$$\lambda^3 - 6\lambda^2 + 5\lambda + 11 = 0. \tag{2.2}$$

By Cayley-Hamilton theorem,

$$A^3 - 6A^2 + 5A + 11I = 0. \tag{2.3}$$

Rearranging,

$$11I = -A^3 + 6A^2 - 5A, \tag{2.4}$$

Solution

and multiplying both sides by A^{-1},

$$A^{-1} = \frac{1}{11}(-A^2 + 6A - 5I) \tag{2.5}$$

$$= \frac{1}{11}\begin{pmatrix} -3 & 4 & 4 \\ 7 & 0 & -3 \\ 5 & -3 & 0 \end{pmatrix}. \tag{2.6}$$

Final Answer:

$$A^{-1} = \begin{pmatrix} -\frac{3}{11} & \frac{4}{11} & \frac{4}{11} \\ \frac{7}{11} & 0 & -\frac{3}{11} \\ \frac{5}{11} & -\frac{3}{11} & 0 \end{pmatrix}. \tag{2.7}$$

C Code

```c
#include <stdio.h>
#include "matfun.h"

void matmult(double A[SIZE][SIZE], double B[SIZE][SIZE], double
    res[SIZE][SIZE]) {
    for(int i=0; i<SIZE; i++) {
        for(int j=0; j<SIZE; j++) {
            res[i][j] = 0.0;
            for(int k=0; k<SIZE; k++) {
                res[i][j] += A[i][k] * B[k][j];
            }
        }
    }
}
```

C Code

```
void matscalarmult(double scalar, double A[SIZE][SIZE], double
    res[SIZE][SIZE]) {
    for(int i=0; i<SIZE; i++) {
        for(int j=0; j<SIZE; j++) {
            res[i][j] = scalar * A[i][j];
        }
    }
}

void matadd(double A[SIZE][SIZE], double B[SIZE][SIZE], double
    res[SIZE][SIZE]) {
    for(int i=0; i<SIZE; i++) {
        for(int j=0; j<SIZE; j++) {
            res[i][j] = A[i][j] + B[i][j];
        }
    }
}
```

C Code

```c
void matsub(double A[SIZE][SIZE], double B[SIZE][SIZE], double
    res[SIZE][SIZE]) {
    for(int i=0; i<SIZE; i++) {
        for(int j=0; j<SIZE; j++) {
            res[i][j] = A[i][j] - B[i][j];
        }
    }
}

void matidentity(double I[SIZE][SIZE]) {
    for(int i=0; i<SIZE; i++) {
        for(int j=0; j<SIZE; j++) {
            I[i][j] = (i == j) ? 1.0 : 0.0;
        }
    }
}
```

C Code

```
void compute_inverse(double A[SIZE][SIZE], double A_inv[SIZE][
    SIZE]) {
    double A2[SIZE][SIZE], negA2[SIZE][SIZE];
    double sixA[SIZE][SIZE], neg5I[SIZE][SIZE];
    double temp[SIZE][SIZE], I[SIZE][SIZE];

    matmult(A, A, A2);
    matscalarmult(-1, A2, negA2);
    matscalarmult(6, A, sixA);

    matidentity(I);
    matscalarmult(-5, I, neg5I);

    matadd(negA2, sixA, temp);
    matadd(temp, neg5I, temp);

    matscalarmult(1.0/11.0, temp, A_inv);
}
```

Python Code

```
iimport ctypes
import numpy as np

SIZE = 3

lib = ctypes.CDLL('./libmatfun.so')

Matrix3x3 = ctypes.c_double * (SIZE*SIZE)

def np_to_c_matrix(np_mat):
    flat = np_mat.flatten()
    return Matrix3x3(*flat)
```

Python Code

```
def c_matrix_to_np(c_mat):
    return np.array(list(c_mat)).reshape((SIZE, SIZE))

# Prototype for compute_inverse
lib.compute_inverse.argtypes = [ctypes.POINTER(ctypes.c_double),
    ctypes.POINTER(ctypes.c_double)]

# Matrix A
A_np = np.array([[1,1,1],
                 [1,2,-2],
                 [2,-1,3]], dtype=np.double)

A_c = np_to_c_matrix(A_np)
A_inv_c = Matrix3x3()
```

Python Code

```
# Call the C function
lib.compute_inverse(A_c, A_inv_c)

A_inv_np = c_matrix_to_np(A_inv_c)

print("Inverse matrix A^-1:")
for row in A_inv_np:
    print(" ".join(f"{x: .4f}" for x in row))
```